Multi-Agent Generative AI System for Business Memo Emailing

Khalil Bou Khalil
Hamadi Azouz
Labiba Farooq

December 2025

1 Introduction

Generative Artificial Intelligence (GenAI) systems are increasingly adopted for professional communication tasks such as drafting business emails and corporate memos. While modern language models can generate fluent text, producing outputs that are consistently clear, well-structured, robust to ambiguity, and trusted by users remains challenging.

This project presents the design and iterative improvement of a multi-agent system named *Business Memo Emailing Crew*. Instead of optimizing only for linguistic fluency, the project follows an evaluation-driven methodology: we first developed a functional minimal viable product (v1.0), evaluated its performance using explicit metrics, identified the main bottleneck, applied targeted improvements, and finally re-evaluated the improved system (v2.0).

During early testing, we observed that although many generated memos were fluent, users frequently rewrote entire sections due to missing information or inappropriate structure. This observation motivated a deeper focus on feedback quality and iterative refinement rather than one-shot generation.

2 System Overview

2.1 Rationale for a Multi-Agent Design

A single-agent approach can generate a memo, but it often conflates requirement analysis, drafting, and self-critique into a single step. In contrast, a multi-agent architecture enforces clear role separation, which improves controllability and makes systematic improvement easier. Each agent can be refined independently without redesigning the entire pipeline.

2.2 System Architecture

The Business Memo Emailing Crew is implemented as a pipeline composed of three specialized agents:

- **Analyst Agent**: interprets the user request, extracts intent, identifies missing information, and formalizes memo constraints.

- **Drafting Agent**: produces an initial memo draft based on the structured analysis.
- **Approval Agent**: reviews the draft, generates feedback, and decides whether refinement or information re-retrieval is required.

The system integrates a lightweight retrieval-augmented generation (RAG) mechanism and operates on a synthetic corporate knowledge base stored in CSV format. A human-in-the-loop interface allows users to approve, reject, or provide feedback. All drafts, feedback, and revisions are logged to enable systematic evaluation across iterations.

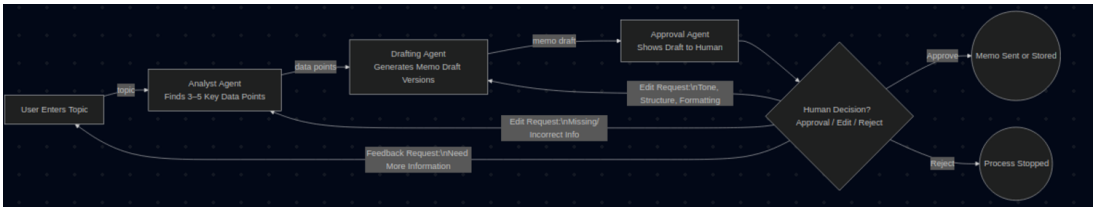

Figure 1: System architecture of the Business Memo Emailing Crew. The pipeline illustrates the interaction between the Analyst, Drafting, and Approval agents, together with retrieval and human-in-the-loop feedback enabling iterative refinement.

Figure 1 summarizes the full workflow and highlights how structured feedback enables controlled iteration rather than one-shot generation.

3 Improvement Hypothesis

Analysis of early results revealed a critical bottleneck: **user feedback was largely unstructured and underspecified**. A large share of feedback consisted of short comments such as "too long" or "missing information", without references to specific sections or actionable guidance.

This led to two main issues:

- The Drafting Agent could not reliably translate feedback into concrete edits.
- The system lacked a trigger to re-retrieve missing information when gaps were identified.

The improvement hypothesis was therefore formulated as follows:

> *If feedback is made structured and actionable, the system will be able to perform targeted refinement and re-retrieval, leading to improvements in memo quality, efficiency, and grounding.*

4 Improvement Actions

To address the identified bottleneck, the following changes were implemented in version v2.0:

- **Structured feedback schema**: the Approval Agent produces section-specific feedback, highlighting missing information, required clarifications, and unnecessary content.
- **Clearer role separation**: agent prompts were refined to reduce overlap and ambiguity.

- **Feedback-driven re-retrieval**: missing or ambiguous information detected during review triggers an additional retrieval step.
- **Preference memory**: recurring user constraints and stylistic preferences are stored to reduce repeated corrections.

These changes transformed the system into an iterative "draft–review–refine" pipeline.

5 Evaluation

Every system run is evaluated immediately after the human approval step. The goal is to track performance over time using structured logs and an aggregate score that balances approval outcome, revision effort, and evidence grounding.

5.1 Evaluation Pipeline

The evaluation pipeline works as follows:

1. `business_memo_system.py` captures run metadata: topic, number of revision cycles, final decision (`approve` / `reject` / `max_cycles_reached`), and whether the final memo was grounded in retrieved evidence.
2. `evaluation_logger.py` appends this record to `evaluation_log.csv` and can also persist it in SQLite (`memo_system.db`).
3. After logging, `calculate_ges.py` can recompute the *Global Evaluation Score (GES)* to monitor trends over time. The helper script `run_ges.py` prints the latest metrics with a single command.

5.2 Logged Fields (per run)

Each run produces one log entry with the following fields:

- `timestamp`: UTC time the run finished
- `topic`: memo subject the user requested
- `revision_cycles`: number of edit loops completed before approval/rejection
- `final_decision`: `approve` / `reject` / `max_cycles_reached` / `unknown`
- `grounded`: `True` if at least one curated data point supported the final draft

5.3 Global Evaluation Score (GES)

To summarize performance across multiple runs, we compute the Global Evaluation Score (GES):

$$GES = 0.5 \cdot approval_rate + 0.3 \cdot \left(1 - \min\left(1, \frac{avg_revisions}{3}\right)\right) + 0.2 \cdot groundedness_rate$$

Where:

- **Approval rate**: proportion of runs that ended in `approve`.
- **Average revisions**: mean revision cycles per run; more than 3 cycles are penalized via $\min(1, \frac{avg_revisions}{3})$.
- **Groundedness rate**: proportion of *approved* memos that were grounded in retrieved evidence.

This weighting (0.5 / 0.3 / 0.2) balances quality, efficiency, and factual support. Higher scores indicate more approvals, fewer revisions, and stronger evidence grounding.

5.4 Why This Formula Fits the Project

- **Approval rate** reflects final human judgment: the multi-agent loop is successful only if the final memo is approved.
- The **revision penalty** captures operational friction: many iterations indicate weaknesses in analysis, drafting, or feedback quality.
- **Groundedness** reinforces a retrieval-first design by rewarding memos supported by curated evidence rather than hallucinated content.
- Normalizing at **three cycles** matches the guardrails in the CLI/Streamlit flows, which stop after roughly three rounds, ensuring the metric reflects real usage constraints.

Together, these signals incentivize improvements that matter most for a business memo assistant: reliable approvals, fast iteration, and evidence-backed content.

5.5 How to Analyze Results

- `python run_ges.py`: quick snapshot of the latest aggregate metrics.
- `python analyze_evaluation.py`: richer statistics (per-topic averages, approval counts, etc.).
- The file `evaluation_log.csv` can be loaded into notebooks or BI tools for deeper analysis.

These tools support both quantitative tracking (GES) and qualitative review (per-run logs and feedback).

6 Quality Assurance

Before pushing changes, we run a quick validation suite to ensure the evaluation logic and pipeline remain correct:

```
# 1. Unit tests (calculates GES edge cases)
pytest

# 2. Recompute the latest evaluation metrics
python run_ges.py
```

The `pytest` suite focuses on `calculate_ges` edge cases (empty logs, all approvals, mixed outcomes) to prevent regressions in scoring logic. The script `run_ges.py` consumes the current `evaluation_log.csv` and prints the aggregate metrics, serving as a sanity check for CSV integrity.

If the human-in-the-loop flow is modified (e.g., `business_memo_system.py` or `streamlit_app.py`), manual runs are also performed because interactive approval input cannot be fully covered by automated tests.

7 AI Assistance Disclosure

- Parts of this repository were authored with the help of an AI pair-programming assistant (e.g., documentation, helper scripts, and generating test data for `evaluation_log.csv`).
- All AI-generated contributions were reviewed, edited, and validated using the tests above to ensure correctness and maintain project style.
- Future iterations should continue documenting AI-assisted contributions so reviewers understand which components were machine-assisted.

8 Challenges and Limitations

Despite improvements, several limitations remain:

- Highly ambiguous requests with missing key details still require careful clarification.
- Human approval and ratings, while informative, remain partially subjective.
- The synthetic knowledge base is limited in size and diversity compared to real corporate environments.
- Grounding checks depend on the quality and coverage of curated CSV evidence.

9 Conclusion and Future Work

This project demonstrates that evaluation-driven iteration is essential for building reliable multi-agent GenAI systems. The system evolved from a basic draft generator into an iterative "draft–review–refine" pipeline with logged evaluation and trend tracking.

Future work includes role-based memo templates (e.g., CEO, HR, Engineering), multilingual support (French and Arabic), stronger grounding verification, and continued emphasis on offline, GDPR-compliant design.